ackee
blockchain security

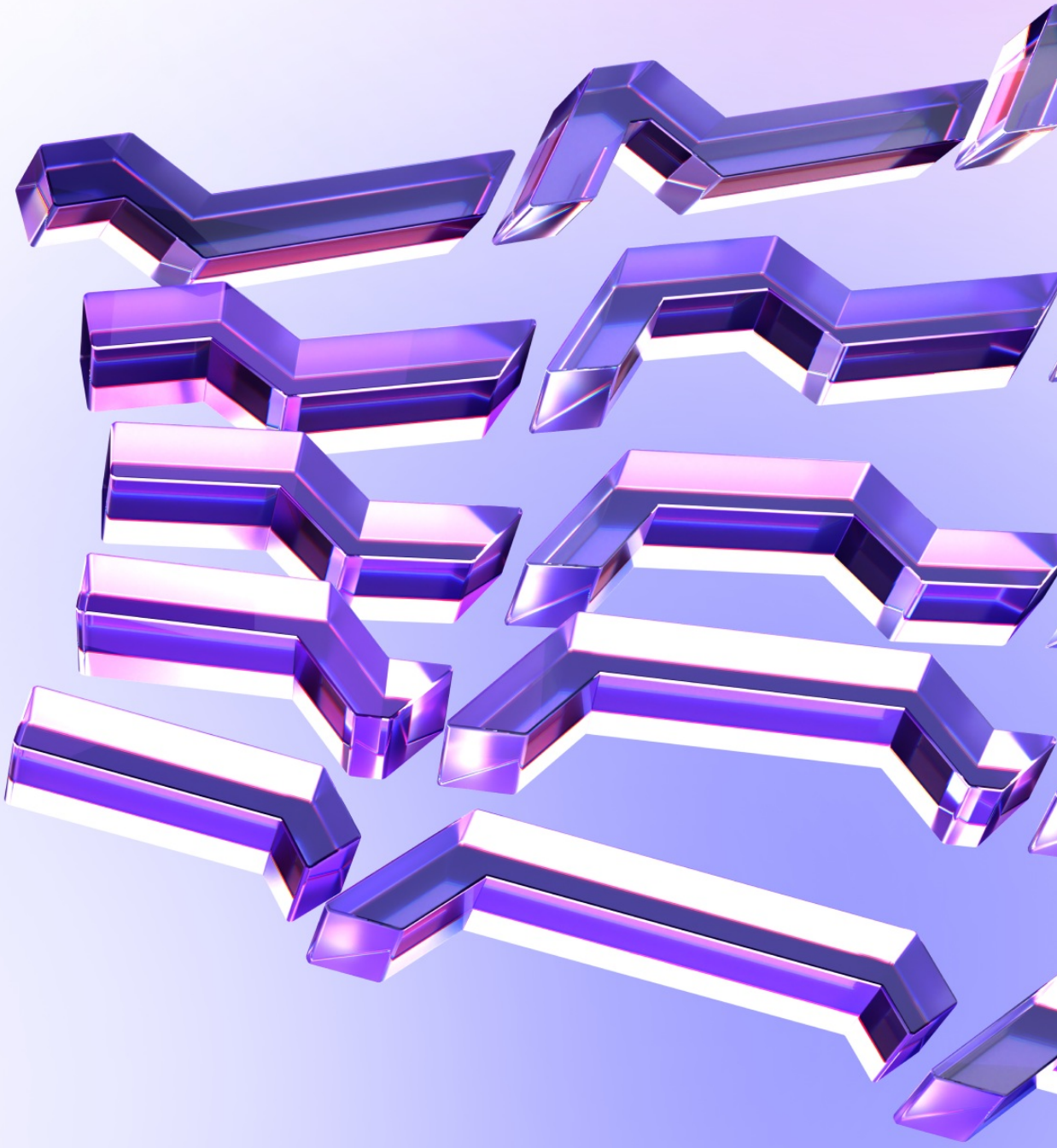
Wake

evgenynacu

usdd-morpho-vault

February 25, 2026 13:22 UTC

Wake Arena AI Report



Contents

1. Document Revisions . . . 3
2. Overview . . . 4
2.1. Ackee Blockchain Security . . . 4
2.2. Wake Arena . . . 5
2.3. Disclaimer . . . 5
2.4. Finding Classification . . . 6
3. Executive Summary . . . 7
Revision 1.0 . . . 7
4. Findings Summary . . . 10
Report Revision 1.0 . . . 12
Findings . . . 12
Appendix A: How to cite . . . 55

1. Document Revisions

1.0	AI Scan	Feb 25, 2026 13:22 UTC

2. Overview

This document presents the findings Wake Arena identified using AI vulnerability checks in the reviewed contracts.

2.1. Ackee Blockchain Security

Ackee Blockchain Security is an in-house team of security researchers performing security audits focusing on manual code reviews with extensive fuzz testing for Ethereum and Solana. Ackee is trusted by top-tier organizations in web3, securing protocols including Lido, Safe, and Axelar.

We develop open-source security and developer tooling Wake for Ethereum and Trident for Solana, supported by grants from Coinbase and the Solana Foundation. Wake and Trident help auditors in the manual review process to discover hardly recognizable edge-case vulnerabilities.

Our team teaches about blockchain security at the Czech Technical University in Prague, led by our co-founder and CEO, Josef Gattermayer, Ph.D. As the official educational partners of the Solana Foundation, we run the School of Solana and the Solana Auditors Bootcamp.

Ackee's mission is to build a stronger blockchain community by sharing our knowledge.

Ackee Blockchain a.s.
Rohanske nabrezi 717/4
186 00 Prague, Czech Republic
https://ackee.xyz
hello@ackee.xyz

2.2. Wake Arena

This report was created by Wake Arena, an automated vulnerability analysis tool. It uses Wake Framework with additional detectors for AI and static analysis of Solidity smart contracts.

To find potential vulnerabilities, Wake uses:

- The code structure and patterns
- Control flow graphs
- Data flow graphs
- Common vulnerability patterns
- Contract interactions

Wake Arena aims for precision, not recall. This means the presented findings are optimized for true positives, not for finding all possible issues, and should always be complemented with additional manual review for a complete security assessment.

2.3. Disclaimer

We've put our best effort to find known vulnerabilities in the system, but automated findings shouldn't be considered a complete list of all existing issues. The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

2.4. Finding Classification

Each finding is classified by two independent ratings: *Impact* and *Confidence*.

Impact

Measuring the potential consequences of the issue on the system.

- **Critical** - Code that activates the issue directly enables theft of significant assets with minimal preconditions.
- **High** - Code that activates the issue will lead to undefined or catastrophic consequences for the system.
- **Medium** - Code that activates the issue will result in consequences of serious substance.
- **Low** - Code that activates the issue will have outcomes on the system that are either recoverable or don't jeopardize its regular functioning.
- **Warning** - The issue represents a potential security concern in the code structure or logic that could become problematic with code modifications.
- **Info** - The issue relates to code quality practices that may affect security. Examples include insufficient logging for critical operations or inconsistent error handling patterns.

Confidence

Indicating the probability that the identified issue is a valid security concern.

- **High** - The analysis has identified a pattern that strongly indicates the presence of the issue.
- **Medium** - Evidence suggests the issue exists, but manual verification is recommended.
- **Low** - Potential indicators of the issue have been detected, but there is a significant possibility of false positives.

3. Executive Summary

Revision 1.0

Audit Overview

This executive summary covers the security review of evgenynacu/usdd-morpho-vault at commit `d128118922752cf3dd44795a87bd4695f5b56619`. The scoped components include the upgradable ERC-4626 vault `SUSDDVault` integrating with Morpho via `IMorpho` and `MarketParamsLib`, the swap/quoting library `SwapHelper` bridging USDT, USDD, and `sUSDD` through a PSM, the administrative `EmergencyKeeper` for batch `rebalance` and `pause` operations, and utility `Constants`.

A total of 13 AI-detected issues were identified across the codebase (3 medium, 8 low, 1 warning, 1 info). The most severe detected issues are concentrated in the `EmergencyKeeper` contract, where unbounded gas forwarding in `idleAll`, `pauseAll`, and `emergencyAll` enables out-of-gas denial-of-service across batch operations. Additional systemic findings include fee-assumption mismatches and fee-unaware previews in `SwapHelper` that can cause optimistic quotes or one-way flows when PSM `tin`/`tout` are non-zero, decimal truncation that strands `USDD` dust and allows zero-output swaps for very small amounts, and a rounding edge case in `SUSDDVault` withdrawals that can underpay tiny redemptions. One informational item notes a documentation mismatch where `MAX_LTV` is set to `0.915e18` (91.5%) while the comment states “90%”.

Overall, the codebase demonstrates solid architecture and reliance on well-known OpenZeppelin upgradable patterns and ERC-4626 semantics. No critical or high-impact issues were detected. The medium-impact items are operational reliability concerns in emergency batch flows rather than direct asset-stealing bugs. Addressing the gas-bounding in `EmergencyKeeper`, hardening `SwapHelper` with fee-awareness and min-out checks, and correcting

rounding/documentation mismatches should materially strengthen resilience and predictability. With these remediations, the security posture is sound for production use; without them, batch emergency actions and swap UX can be fragile under adverse conditions or governance-fee changes.

Key Technical Findings

- Unbounded gas forwarding in `EmergencyKeeper.idleAll`, `pauseAll`, and `emergencyAll` allows a single adversarial or pathological vault to consume nearly all gas and revert, starving subsequent iterations and causing batch DoS. Bound per-call gas using low-level `address(v).call{gas: GAS_LIMIT}(…)` and record success booleans, or split batches to isolate problematic targets.
- Proportional withdrawals in `SUSDDVault._unwindPosition` require both `sharesToRepay > 0` and `collateralToWithdraw > 0`; for tiny `withdrawRatio`, integer division can zero one or both, skipping the unwind after shares are burned and underpaying the redeemer. Enforce a minimum redeem threshold, round components up with caps, or batch small redemptions to avoid silent value leakage.
- PSM fee assumption in `SwapHelper` is misleading: `swapUSDTtoSUSDD` continues to execute under `tin > 0` (reduced proceeds), while `swapSUSDDtoUSDT` and `swapUSDDtoUSDT` will revert under `tout > 0` due to approving only net USDD. Either enforce `psm.tin()==0 && psm.tout()==0` at runtime or compute fee-inclusive approvals and add caller-specified min-out.
- Preview helpers `previewSwapUSDTtoSUSDD`, `previewSwapSUSDDtoUSDT`, and `previewSUSDDNeededForUSDT` ignore `IPSM.tin`/`IPSM.tout`, yielding optimistic quotes when fees are non-zero. Make previews fee-aware or revert when fees are enabled to keep off-chain quoting accurate.
- Decimal truncation in `swapSUSDDtoUSDT` and `swapUSDDtoUSDT` (`/ 1e12`) strands `USDD` dust on the calling contract and allows zero-output swaps for small inputs. Add minimum-amount checks, track and sweep leftovers, or return

the remainder so integrators can reconcile balances.

- Documentation drift: `MAX_LTV` is `0.915e18` while the inline comment says "90% max (below LLTV)". Align code and comments or derive the cap from `marketParams.lltv` minus a configured safety buffer.

Conclusion

The absence of critical and high-impact findings, combined with a clear separation of concerns and conventional upgradable/ ERC-4626 design, indicates a mature baseline. However, the three medium-impact issues in `EmergencyKeeper` present a tangible batch-processing DoS risk that should be remediated before production reliance on one-click emergency operations. Low-severity items in `SwapHelper` and `SUSDDVault` are straightforward to address with fee-awareness, min-out constraints, dust handling, and clearer parameterization.

With these targeted fixes and operational runbooks in place, the project is suitable for deployment. Deferring the mitigations would leave emergency flows and quoting behavior brittle under fee changes or adversarial conditions, so final readiness depends on resolving the medium-severity items and adopting the recommended safeguards.

4. Findings Summary

Summary of findings:

Critical	High	Medium	Low	Warning	Info	Total
0	0	3	8	1	1	**13**

Table 1. Findings Count by Impact

Findings in detail:

Finding title	Impact	Reported	Status
M1: Unbounded gas forwarding in pauseAll enables out-of-gas denial of service	Medium	1.0	Reported
M2: Unbounded gas forwarding in emergencyAll enables gas-griefing across both rebalance and pause	Medium	1.0	Reported
M3: Unbounded gas forwarding in idleAll enables out-of-gas denial of service	Medium	1.0	Reported
L1: swapUSDTtoSUSDD lacks `tin == 0` enforcement; non-zero PSM sell fee silently reduces proceeds	Low	1.0	Reported
L2: Rounding in proportional withdrawals can underpay small share redemptions	Low	1.0	Reported
L3: previewSwapUSDTtoSUSDD ignores PSM `tin`, overestimates sUSDD	Low	1.0	Reported

Finding title	Impact	Reported	Status
L4: All preview helpers ignore PSM fees, returning optimistic quotes under non-zero `tin`/`tout`	Low	1.0	Reported
L5: previewSwapSUSDDtoUSDT ignores PSM `tout`, overestimates USDT	Low	1.0	Reported
L6: Decimal truncation in swapSUSDDtoUSDT leaves USDD dust and can return 0 USDT	Low	1.0	Reported
L7: Decimal truncation in swapUSDDtoUSDT leaves USDD dust and can return 0 USDT	Low	1.0	Reported
L8: previewSUSDDNeededForUSDT ignores PSM `tout`, underestimates required sUSDD	Low	1.0	Reported
W1: Misleading PSM-fee assumption: deposits via sellGem still execute under tin>0 while redemptions via buyGem revert under tout>0	Warning	1.0	Reported
I1: Documentation mismatch: MAX_LTV is 0.915e18 (91.5%) but comment states 90%	Info	1.0	Reported

Table 2. Table of Findings

Report Revision 1.0

Findings

The following section presents the list of findings discovered in this revision.

For the complete list of all findings, Go back to Findings Summary

M1: Unbounded gas forwarding in pauseAll enables out-of-gas denial of service

Medium impact finding

Impact:	Medium	Confidence:	Medium
Target:	./contracts/EmergencyKeeper.sol	Type:	Denial of service

Description

The function loops over user-supplied vault addresses and makes a high-level external call `IVault(vaults[i]).pause()` inside `try/catch`. By default, such calls forward nearly all remaining gas to the callee (EIP-150 only preserves ~1/64). A malicious or compromised vault can burn the forwarded gas before reverting, and although the revert is caught, the caller's remaining gas is severely depleted. This can prevent emitting events and/or processing subsequent vaults in the same batch, effectively producing a denial of service for the intended "pause many" operation.

Affected components:

- Function `pauseAll` (batch pause across vaults); and
- Event `PauseResult` emitted per target.

Listing 1. Code snippet from contracts/EmergencyKeeper.sol

```
function pauseAll(
    address[] calldata vaults
) external onlyOwner returns (bool[] memory ok) {
    ok = new bool[](vaults.length);
    for (uint256 i = 0; i < vaults.length; i++) {
        try IVault(vaults[i]).pause() {
            ok[i] = true;
        } catch {}
        emit PauseResult(vaults[i], ok[i]);
    }
```

```
}
```

Exploit Scenario

A single adversarial vault can cause the transaction to run out of gas so that other vaults are not paused.

Consider the following attack scenario:

1. Alice controls a vault whose `pause` implementation deliberately consumes substantial gas before reverting.
2. Bob, the owner, invokes `pauseAll` with a list of vaults including Alice's and several legitimate vaults.
3. The call to `IVault(AliceVault).pause()` forwards almost all remaining gas; Alice's implementation consumes it, then reverts. The `try/catch` catches the error but cannot recover the burned gas.
4. The remaining gas is insufficient to continue iterating and emitting events for the rest of the list, causing the transaction to revert out-of-gas or to stop early without pausing later vaults.

Recommendation

Bound the worst-case gas consumption per external call to prevent one target from starving the entire loop.

Fix the vulnerability by either:

- Using a low-level call with a per-call gas limit, e.g. `(bool success, ) = address(v).call{gas: GAS_LIMIT}(abi.encodeWithSelector(IVault.pause.selector));`, then emit `PauseResult(v, success)`.
- Processing smaller batches or isolating suspicious vaults into distinct transactions.

- Avoiding any heavy work immediately after a potentially adversarial call so that minimal gas is needed to record the per-target result.

Go back to Findings Summary

M2: Unbounded gas forwarding in emergencyAll enables gas-griefing across both rebalance and pause

Medium impact finding

Impact:	Medium	Confidence:	Medium
Target:	./contracts/EmergencyKeeper.sol	Type:	Denial of service

Description

The function performs two high-level external calls per vault - first `rebalance(IDLE_MODE)`, then `pause` - inside a single loop. Each external call forwards nearly all remaining gas to the callee by default (EIP-150 retains only ~1/64), so a malicious or pathological vault can consume almost all of the gas and revert. Even though failures are caught via `try/catch`, the spent gas is lost, commonly leaving insufficient gas for emitting result events and for making the subsequent external call(s) or processing the next vault. This enables a denial-of-service that undermines the "one-click emergency" behavior across a batch.

Affected components:

- Function `emergencyAll` (sequential `rebalance` then `pause` per vault);
- Constant `IDLE_MODE` used by `rebalance`; and
- Events `IdleResult` and `PauseResult` emitted per target.

Listing 2. Code snippet from contracts/EmergencyKeeper.sol

```
function emergencyAll(address[] calldata vaults) external onlyOwner {
    for (uint256 i = 0; i < vaults.length; i++) {
        try IVault(vaults[i]).rebalance(IDLE_MODE) {
            emit IdleResult(vaults[i], true);
        } catch {
```

```
            emit IdleResult(vaults[i], false);
        }
        try IVault(vaults[i]).pause() {
            emit PauseResult(vaults[i], true);
        } catch {
            emit PauseResult(vaults[i], false);
        }
    }
}
```

Exploit Scenario

One adversarial vault in the list can starve the transaction of gas, blocking the remaining emergency actions.

Consider the following attack scenario:

1. Alice controls a vault that implements `IVault` but whose `rebalance` and/or `pause` deliberately burns most of the gas before reverting.
2. Bob, the owner, calls `emergencyAll` with a list of vaults including Alice's followed by legitimate vaults.
3. The first external call to Alice's vault forwards nearly all remaining gas and is gas-burned; the error is caught but the caller's gas is mostly depleted. The subsequent call to `pause` and the processing of later vaults fail due to insufficient remaining gas.
4. Other vaults in the same transaction do not receive their emergency `rebalance`/`pause` operations, delaying or preventing the intended shutdown across the system.

Recommendation

Defend the batch against gas-griefing by bounding gas per external call and minimizing post-call assumptions about remaining gas.

Fix the vulnerability by either:

- Performing low-level calls with an explicit per-call gas stipend, recording success booleans from the returned status, and emitting the corresponding events.
- Splitting the operation into smaller batches or separate ”idle” and ”pause” transactions when interacting with potentially problematic targets.
- Introducing a configurable per-call gas limit that can be tuned on deployment per network, with safe defaults and clear operational runbooks.

Go back to Findings Summary

M3: Unbounded gas forwarding in idleAll enables out-of-gas denial of service

Medium impact finding

Impact:	Medium	Confidence:	Medium
Target:	./contracts/EmergencyKeeper.sol	Type:	Denial of service

Description

The function forwards essentially all remaining gas to an external call inside a loop. Each iteration calls `IVault(vaults[i]).rebalance(IDLE_MODE)` using a high-level external call, which by default forwards nearly all gas to the callee (EIP-150 only retains ~1/64 for the caller). A malicious or pathological vault implementation can deliberately consume almost all forwarded gas before reverting. Although the revert is caught by `try/catch`, the spent gas is not recovered, leaving too little gas for the rest of the loop and subsequent iterations. This creates a denial-of-service vector that can prevent reliable processing of other vaults in the batch in the same transaction.

Affected components:

- Function `idleAll` (batch rebalance to `IDLE_MODE`);
- Constant `IDLE_MODE` used as the `rebalance` argument; and
- Event `IdleResult` emitted per target.

Listing 3. Code snippet from contracts/EmergencyKeeper.sol

```
function idleAll(
    address[] calldata vaults
) external onlyOwner returns (bool[] memory ok) {
    ok = new bool[](vaults.length);
    for (uint256 i = 0; i < vaults.length; i++) {
        try IVault(vaults[i]).rebalance(IDLE_MODE) {
```

```
            ok[i] = true;
        } catch {}
        emit IdleResult(vaults[i], ok[i]);
    }
}
```

Exploit Scenario

A malicious vault can gas-grief the batch so that subsequent vaults are not processed.

Consider the following attack scenario:

1. Alice deploys or controls a vault address that implements `IVault` but where `rebalance` performs heavy computation or a gas-burning loop before reverting.
2. Bob, the contract owner, calls `idleAll` with a list of vaults that includes Alice's vault together with legitimate vaults.
3. The call `IVault(AliceVault).rebalance(IDLE_MODE)` forwards almost all remaining gas; Alice's implementation consumes it and reverts. The `try/catch` records failure but the gas is already depleted.
4. With too little gas left to continue the loop and emit events for later entries, the transaction either reverts out-of-gas or fails to process remaining vaults, preventing timely emergency handling for them.

Recommendation

Prefer bounding the gas forwarded to each external call so one misbehaving target cannot starve the entire transaction.

Fix the vulnerability by either:

- Invoking the vault via a low-level call with an explicit gas stipend, and treating failure as `false`: for example, `(bool success, ) = address(v).call{gas:`

`GAS_LIMIT}(abi.encodeWithSelector(IVault.rebalance.selector, IDLE_MODE));` then emit `IdleResult(v, success)`.

- Limiting batch size and/or isolating problematic vaults into separate transactions so a single target cannot block the rest.
- Minimizing per-iteration work that assumes ample remaining gas; avoid performing additional heavy logic immediately after adversarial calls.

Go back to Findings Summary

L1: swapUSDTtoSUSDD lacks `tin == 0` enforcement; non-zero PSM sell fee silently reduces proceeds

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/libraries/SwapHelper.sol	Type:	Configuration

Description

The `swapUSDTtoSUSDD` execution path approves USDT to the PSM `gemJoin`, calls `psm.sellGem`, measures the USDD delta, and deposits that entire amount into the `sUSDD` ERC-4626 vault. If PSM `tin > 0`, `sellGem` withholds a fee, yet the function continues normally and simply mints fewer `sUSDD` shares. This contradicts the file-level comment that states swap functions will revert if PSM fees are enabled, and there is no `min-out`/slippage guard to prevent silent value loss.

Affected elements:

- Function `swapUSDTtoSUSDD` in `contracts/libraries/SwapHelper.sol`;
- External parameter `IPSM.tin` exposed by `contracts/interfaces/IPSM.sol`.

Listing 4. Code snippet from contracts/libraries/SwapHelper.sol (documented assumption)

```
/// PSM Fee Assumption:
/// This library assumes PSM tin/tout fees are 0 (current mainnet state).
/// If PSM governance enables fees, swap functions will revert and the vault
/// will need to be upgraded. This simplifies code and saves gas.
```

Listing 5. Code snippet from contracts/libraries/SwapHelper.sol (execution path)

```
        // Step 1: USDT -> USDD via PSM
        address gemJoin = psm.gemJoin();
        usdt.forceApprove(gemJoin, usdtAmount);

        // sellGem: sends USDT, receives USDD (1:1 when tin=0)
        uint256 usddBalanceBefore = usdd.balanceOf(address(this));
        psm.sellGem(address(this), usdtAmount);
        uint256 usddReceived = usdd.balanceOf(address(this)) -
            usddBalanceBefore;

        // Step 2: USDD -> sUSDD via staking
        usdd.forceApprove(Constants.SUSDD, usddReceived);
        susddAmount = susdd.deposit(usddReceived, address(this));
```

Because `sellGem` with `tin > 0` does not revert (it reduces the amount of USDD returned instead), the current implementation will silently produce fewer `sUSDD` than expected whenever fees are active, directly conflicting with the documented revert expectation.

Exploit Scenario

How the mismatch materializes in practice:

Consider the following attack scenario:

1. Alice (governance) enables a non-zero `tin` on the PSM or deploys a replacement PSM with `tin > 0`;
2. Bob submits a USDT->sUSDD swap through a contract using this helper, expecting the historical 1:1 behavior or a revert on fee activation as documented;
3. `psm.sellGem` withholds the `tin` fee and returns fewer USDD to the helper; and
4. The helper deposits the reduced USDD into the vault, minting fewer `sUSDD`

without any revert or min-out protection, causing Bob silent value loss relative to expectations.

Recommendation

Align code with the documented assumption and add safety controls:

- Enforce the zero-fee invariant at runtime before executing the swap:
- `require(IPSM(Constants.PSM).tin() == 0, "PSM sell fee enabled");`
- Additionally, add a caller-specified `minSUSDDOut` parameter to the swap and revert if the minted shares fall below it, e.g.:
- `function swapUSDTtoSUSDD(uint256 usdtAmount, uint256 minSUSDDOut) internal returns (uint256) { … require(susddAmount >= minSUSDDOut, "slippage"); }`
- If supporting non-zero fees is desired, compute the expected `usddReceived` using `tin` and enforce the min-out check accordingly.

Document these behaviors clearly so integrators are not misled during future fee changes.

Go back to Findings Summary

L2: Rounding in proportional withdrawals can underpay small share redemptions

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/SUSDDVault.sol	Type:	Logic error

Description

A proportional-withdrawal implementation in `redeem` and `_unwindPosition` can underpay very small redemptions due to integer rounding. When the vault has an active Morpho position, the code computes proportional components `sharesToRepay` and `collateralToWithdraw`. If either rounded component is zero while debt exists, the function skips unwinding the position and transfers only the idle USDT portion. Because shares are already burned, the redeemer does not receive any of their pro-rata equity embedded in the position; this residual value remains in the vault and benefits remaining holders.

The behavior hinges on both proportional components being strictly greater than zero to execute the flash-loan unwind path. For tiny `withdrawRatio`, both `sharesToRepay` and `collateralToWithdraw` can round to zero.

Listing 6. Code snippet from contracts/SUSDDVault.sol

```
function _unwindPosition(
    uint256 withdrawRatio,
    address receiver
) internal returns (uint256) {
    uint256 balanceBefore = IERC20(Constants.USDT).balanceOf(address(this));
    uint256 idleToWithdraw = (balanceBefore * withdrawRatio) /
        Constants.WAD;

    IMorpho morpho = IMorpho(Constants.MORPHO);
    Position memory pos = morpho.position(
        Id.wrap(Constants.MARKET_ID),
```

```
        address(this)
    );

    // Proportionally unwind position (if exists)
    if (pos.collateral > 0) {
        uint256 sharesToRepay = (uint256(pos.borrowShares) *
            withdrawRatio) / Constants.WAD;
        uint256 collateralToWithdraw = (uint256(pos.collateral) *
            withdrawRatio) / Constants.WAD;

        if (sharesToRepay > 0 && collateralToWithdraw > 0) {
            // Both debt and collateral to withdraw - use flash loan
            (
                ,
                ,
                uint256 totalBorrowAssets,
                uint256 totalBorrowShares
            ) = morpho.expectedMarketBalances(marketParams);
            uint256 flashLoanAmount = sharesToRepay.toAssetsUp(
                totalBorrowAssets,
                totalBorrowShares
            );

            bytes memory data = abi.encode(
                OP_WITHDRAW,
                sharesToRepay,
                collateralToWithdraw
            );
            morpho.flashLoan(Constants.USDT, flashLoanAmount, data);
        } else if (collateralToWithdraw > 0 && pos.borrowShares == 0) {
            // No debt at all, safe to just withdraw collateral
            morpho.withdrawCollateral(
                marketParams,
                collateralToWithdraw,
                address(this),
                address(this)
            );
            SwapHelper.swapSUSDDtoUSDT(collateralToWithdraw);
        }
    }

    // Transfer proportional idle + net gain from position unwind
    uint256 balanceAfter = IERC20(Constants.USDT).balanceOf(address(this));
    uint256 gainFromPosition = balanceAfter > balanceBefore
        ? balanceAfter - balanceBefore
        : 0;
```

```
    uint256 toTransfer = idleToWithdraw + gainFromPosition;
    IERC20(Constants.USDT).safeTransfer(receiver, toTransfer);

    return toTransfer;
}
```

In `redeem`, the shares are burned prior to calling `_unwindPosition`, so any skipped unwind directly translates into value leakage from the redeemer to remaining holders.

Listing 7. Code snippet from contracts/SUSDDVault.sol

```
// Calculate withdrawal ratio BEFORE burning (shares-based, no NAV needed)
uint256 withdrawRatio = (shares * Constants.WAD) / supplyBefore;

// Burn shares
_burn(owner, shares);

// Unwind position and get actual USDT amount
uint256 assets = _unwindPosition(withdrawRatio, receiver);
```

Conditions for rounding to zero follow from integer division:

- `sharesToRepay = floor(pos.borrowShares * withdrawRatio / WAD)` becomes zero when `withdrawRatio < WAD / pos.borrowShares`.
- `collateralToWithdraw = floor(pos.collateral * withdrawRatio / WAD)` becomes zero when `withdrawRatio < WAD / pos.collateral`.

Therefore, with large on-chain position values and small `shares`, it is realistic for both components to round to zero. In that case, the contract transfers only `idleToWithdraw` (which itself can be zero when idle liquidity is low), while the redeemer's shares have already been burned.

Impact in production is low but concrete: very small redemptions can be underpaid by up to the redeemer's pro-rata share of the net position equity for that call. The foregone value remains in the vault and accrues to remaining LPs. Repeated small redemptions by integrators or automated withdrawal

flows can accumulate non-trivial residuals over time.

Exploit Scenario

A realistic exploitation path leverages the rounding edge cases to make small redeemers underpaid while residual value remains with other holders.

Consider the following attack scenario:

1. Alice, a large LP, monitors that the vault holds a sizable leveraged position with minimal idle USDT.
2. Bob, a user or an integrator, submits a very small `redeem` that produces a tiny `withdrawRatio`.
3. The contract computes `sharesToRepay` and `collateralToWithdraw`, both round to zero, so `_unwindPosition` skips unwinding and returns only `idleToWithdraw`.
4. Bob's shares are already burned, Bob receives little or nothing, and the residual equity remains in the vault, benefiting Alice and other remaining LPs.

Recommendation

Mitigate by ensuring that proportional withdrawals cannot silently skip the position unwind when `pos.borrowShares > 0`:

Fix the vulnerability by either:

- Enforcing a minimum redeem amount when debt exists. Compute the minimum `withdrawRatio` that makes both components non-zero and revert below it, e.g.:
- `minRatio = max(ceilDiv(WAD, pos.borrowShares), ceilDiv(WAD, pos.collateral))`;
- Require `withdrawRatio >= minRatio` if `pos.borrowShares > 0`.

- Rounding up the proportional components and capping to available amounts, then using the flash-loan path:
- `sharesToRepay = min(pos.borrowShares, mulDivUp(pos.borrowShares, withdrawRatio, WAD));`
- `collateralToWithdraw = min(pos.collateral, mulDivUp(pos.collateral, withdrawRatio, WAD));`

  ```
  This guarantees at least one unit of each component when `withdrawRatio > 0`, triggering the unwind. To avoid aggregate positive drift from rounding up across many redeemers, account for rounding dust explicitly (e.g., track a `roundingDust` bucket or charge a negligible fee that is redistributed fairly).
  ```

- Alternatively, batch or queue small redemptions: accumulate requests until the aggregate ratio crosses the minimum threshold, then process in one unwind operation.

Additionally:

- Document the behavior in `previewRedeem`/`previewWithdraw` and enforce the same minimums there to give integrators deterministic outcomes.
- If retaining the current behavior, add a clear revert such as `RedeemTooSmall()` to prevent silent value leakage when the position exists.

[Go back to Findings Summary]

L3: previewSwapUSDTtoSUSDD ignores PSM `tin`, overestimates sUSDD

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/libraries/SwapHelper.sol	Type:	Logic error

Description

The `previewSwapUSDTtoSUSDD` path assumes a fee-less 1:1 USDT->USDD conversion at the PSM and therefore multiplies the USDT amount by `1e12` before calling `susdd.previewDeposit`. When PSM `tin > 0`, `sellGem` withholds a fee from the USDD credited to the caller, so the preview returns a larger `sUSDD` amount than what can actually be minted at execution time.

Affected elements:

- Function `previewSwapUSDTtoSUSDD` in `contracts/libraries/SwapHelper.sol`;
- External parameter `IPSM.tin` exposed by `contracts/interfaces/IPSM.sol` (not queried here).

This creates optimistic quotes that can mislead integrators relying on previews for slippage bounds or off-chain RFQs. In production, a governance change that enables non-zero `tin` (or a deployment to an environment where `tin != 0`) will immediately invalidate the preview without any runtime detection.

Listing 8. Code snippet from contracts/libraries/SwapHelper.sol

```
        // USDT -> USDD (1:1 when tin=0)
        uint256 usddAmount = usdtAmount * 1e12;

        // USDD -> sUSDD
```

```
        susddAmount = susdd.previewDeposit(usddAmount);
```

Exploit Scenario

A realistic sequence under non-zero PSM fees:

Consider the following attack scenario:

1. Alice (governance) enables a non-zero `tin` fee on the PSM;
2. Bob calls a contract that uses `previewSwapUSDTtoSUSDD` to estimate how many `sUSDD` he will receive for 100 USDT;
3. The preview reports an amount computed from `100e6 * 1e12` USDD with no fee, then `susdd.previewDeposit` of that value; and
4. When Bob later executes the actual swap, `sellGem` withholds `tin`, fewer USDD are obtained, and fewer `sUSDD` are minted than previewed, causing slippage or failed expectations.

Recommendation

Make the preview fee-aware or fail fast when the assumption is violated.

Two complementary approaches:

- Query `tin` and adjust the USDD passed into the ERC-4626 preview:
- `uint256 tin = IPSM(Constants.PSM).tin();`
- `uint256 usddGross = usdtAmount * 1e12;`
- `uint256 usddNet = usddGross - (usddGross * tin / 1e18);`
- `return IERC4626(Constants.SUSDD).previewDeposit(usddNet);`
- Alternatively, enforce the invariant and surface the assumption explicitly:
- `require(IPSM(Constants.PSM).tin() == 0, "PSM fee enabled");`

In either case, document the rounding strategy and consider adding a safety

margin when converting between 6 and 18 decimals.

Go back to Findings Summary

L4: All preview helpers ignore PSM fees, returning optimistic quotes under non-zero `tin`/`tout`

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/libraries/SwapHelper.sol	Type:	Data validation

Description

The three preview helpers perform straight 6↔18 decimal scaling around ERC-4626 previews under the stated assumption that PSM `tin`/`tout` are zero. None of them queries `IPSM.tin` or `IPSM.tout`. When fees are enabled, previews will systematically deviate from execution results:

- `previewSwapUSDTtoSUSDD`: overestimates `sUSDD` out because `sellGem` returns `USDD - tin`;
- `previewSwapSUSDDtoUSDT`: overestimates `USDT` out because `buyGem` requires `USDD + tout`;
- `previewSUSDDNeededForUSDT`: underestimates `sUSDD` required to reach a target `USDT` because extra `USDD` is needed to pay `tout`.

Affected elements:

- Functions `previewSwapUSDTtoSUSDD`, `previewSwapSUSDDtoUSDT`, and `previewSUSDDNeededForUSDT` in `contracts/libraries/SwapHelper.sol`.

These optimistic previews can mislead off-chain quoting, frontends, and slippage protections in integrators that treat previews as authoritative.

Listing 9. Code snippet from contracts/libraries/SwapHelper.sol (USDT -> sUSDD preview)

```
        // USDT -> USDD (1:1 when tin=0)
        uint256 usddAmount = usdtAmount * 1e12;

        // USDD -> sUSDD
        susddAmount = susdd.previewDeposit(usddAmount);
```

Listing 10. Code snippet from contracts/libraries/SwapHelper.sol (sUSDD -> USDT preview)

```
        // sUSDD -> USDD
        uint256 usddAmount = susdd.previewRedeem(susddAmount);

        // USDD -> USDT (1:1 when tout=0)
        usdtAmount = usddAmount / 1e12;
```

Listing 11. Code snippet from contracts/libraries/SwapHelper.sol (sUSDD needed for target USDT)

```
        // USDT -> USDD needed (1:1 when tout=0)
        uint256 usddNeeded = usdtAmount * 1e12;

        // How many sUSDD shares to burn for this USDD amount
        susddAmount = susdd.previewWithdraw(usddNeeded);
```

Exploit Scenario

How previews diverge from reality when fees are active:

Consider the following attack scenario:

1. Alice (governance) enables non-zero `tin`/`tout` on the PSM after a dApp integrates these previews for quoting and slippage checks;
2. Bob relies on `preview*` outputs to price a swap and to set tight min-out thresholds;

3. The previews were computed without fees, so Bob's transaction either reverts due to slippage or executes with fewer proceeds than quoted; and
4. The discrepancy leads to financial loss or failed UX for Bob and his integrator.

Recommendation

Provide fee-aware previews or enforce invariants so that quotes remain accurate:

- Query `IPSM.tin()`/`IPSM.tout()` and incorporate them into preview math, e.g.:
- USDT→sUSDD: `usddNet = usdt * 1e12 * (1e18 - tin) / 1e18;` then `previewDeposit(usddNet)`.
- sUSDD→USDT: `usdtOut = (usdd * 1e18) / (1e12 * (1e18 + tout));` where `usdd = previewRedeem(shares)`.
- sUSDD needed for target USDT: `usddNeeded = usdt * 1e12 * (1e18 + tout) / 1e18;` then `previewWithdraw(usddNeeded)`.
- If fee support is out of scope, fail fast to keep previews trustworthy:
- `require(IPSM(Constants.PSM).tin() == 0 && IPSM(Constants.PSM).tout() == 0, "PSM fees enabled");`

Document rounding considerations and communicate that previews are estimates subject to fee and exchange-rate changes.

Go back to Findings Summary

L5: previewSwapSUSDDtoUSDT ignores PSM `tout`, overestimates USDT

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/libraries/SwapHelper.sol	Type:	Logic error

Description

The `previewSwapSUSDDtoUSDT` path assumes a fee-less 1:1 USDD->USDT conversion at the PSM. It previews USDD via `susdd.previewRedeem(susddAmount)` and then divides by `1e12` to obtain a USDT amount. When PSM `tout > 0`, `buyGem` requires additional USDD beyond the target `gemAmt` of USDT, so the maximum obtainable `USDT` is strictly less than `usddAmount / 1e12`.

Affected elements:

- Function `previewSwapSUSDDtoUSDT` in `contracts/libraries/SwapHelper.sol`;
- External parameter `IPSM.tout` exposed by `contracts/interfaces/IPSM.sol` (not queried here).

This produces optimistic quotes for USDT out and can cause integrators to set overly aggressive min-out thresholds or misprice RFQs.

Listing 12. Code snippet from contracts/libraries/SwapHelper.sol

```
        // sUSDD -> USDD
        uint256 usddAmount = susdd.previewRedeem(susddAmount);

        // USDD -> USDT (1:1 when tout=0)
        usdtAmount = usddAmount / 1e12;
```

Exploit Scenario

A realistic sequence under non-zero PSM fees:

Consider the following attack scenario:

1. Alice (governance) enables or raises a non-zero `tout` fee on the PSM;
2. Bob relies on `previewSwapSUSDDtoUSDT` to quote how many USDT he will receive for redeeming 1 sUSDD;
3. The preview divides the USDD preview by `1e12` without accounting for `tout`; and
4. When Bob executes the swap, `psm.buyGem` consumes extra USDD for the fee, so Bob receives fewer USDT than previewed.

Recommendation

Make the preview fee-aware or fail fast when the assumption is violated.

Two complementary approaches:

- Query `tout` and solve for the USDT obtainable from a fixed USDD balance:
- `uint256 tout = IPSM(Constants.PSM).tout();`
- `uint256 usddAvail = IERC4626(Constants.SUSDD).previewRedeem(susddAmount);`
- `uint256 usdtOut = (usddAvail * 1e18) / (1e12 * (1e18 + tout));`
- Alternatively, enforce the invariant and surface the assumption explicitly:
- `require(IPSM(Constants.PSM).tout() == 0, "PSM fee enabled");`

When implementing the division, consider using a checked `mulDiv` to minimize rounding error and document that previews are best-effort estimates.

Go back to Findings Summary

L6: Decimal truncation in swapSUSDDtoUSDT leaves USDD dust and can return 0 USDT

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/libraries/SwapHelper.sol	Type:	Logic error

Description

Decimal truncation when converting 18-decimal `USDD` to 6-decimal `USDT` leads to stranded `USDD` dust and a zero-output edge case.

Function `swapSUSDDtoUSDT` redeems `sUSDD` to `USDD` (18 decimals), then computes `gemAmt = usddReceived / 1e12` to call `psm.buyGem`. Because `gemAmt` is rounded down to 6 decimals, the PSM only consumes `gemAmt * 1e12` wei-USDD. Any remainder `r = usddReceived % 1e12` stays on the calling contract and is never swept by the library. For `usddReceived < 1e12`, `gemAmt == 0`, the PSM call mints `0 USDT`, and the redeemed `USDD` remains on the contract.

Affected surface:

- Function `swapSUSDDtoUSDT` in `contracts/libraries/SwapHelper.sol`.
- External calls to `susdd.redeem` and `psm.buyGem` (confirmed by Wake `get_state_changes`).
- Token balances affected: `USDD.balanceOf(address(this))` of the calling contract accumulates dust.

This creates accounting drift in integrators unless dust is periodically swept or refunded.

Listing 13. Code snippet from contracts/libraries/SwapHelper.sol

```
        // Step 2: USDD -> USDT via PSM
        usdd.forceApprove(Constants.PSM, usddReceived);

        // When tout=0: gemAmt = usddReceived (scaled to 6 decimals)
        uint256 gemAmt = usddReceived / 1e12;

        uint256 usdtBalanceBefore = usdt.balanceOf(address(this));
        psm.buyGem(address(this), gemAmt);
        usdtAmount = usdt.balanceOf(address(this)) - usdtBalanceBefore;
```

Exploit Scenario

An external caller can deliberately trigger dust accumulation and zero-output swaps without violating protocol invariants.

Consider the following attack scenario:

1. Alice calls `swapSUSDDtoUSDT` repeatedly with `sUSDD` amounts that redeem to `usddReceived = k * 1e12 + r` where `0 < r < 1e12`.
2. The function computes `gemAmt = usddReceived / 1e12 = k` and calls `psm.buyGem` for `k` units of `USDT`.
3. Only `k * 1e12` wei-USDD are consumed; the remainder `r` is left on the contract as `USDD` dust.
4. Alice continues this pattern to accumulate non-trivial `USDD` balances on the contract that are not returned to her.
5. Bob, the protocol operator, observes growing `USDD` dust on the contract, affecting accounting/NAV unless a manual sweep is performed.

A second edge case:

1. Alice submits a very small `sUSDD` amount so that `usddReceived < 1e12`.
2. The function computes `gemAmt = 0` and mints `0 USDT`.

3. The full `usddReceived` remains on the contract, creating dust with no user-facing output.

Recommendation

Adopt explicit dust-handling and input-sanity mechanisms around the 18-to-6 decimal conversion.

Fix the vulnerability by either:

- Enforcing a non-zero output: add `require(usddReceived >= 1e12, "Insufficient amount");` or `require(gemAmt > 0, "Insufficient amount");` to prevent zero-output swaps.
- Refunding or sweeping the remainder: after `psm.buyGem`, compute the leftover `USDD` on the contract and either:
- Transfer any remainder `r = USDD.balanceOf(address(this)) % 1e12` back to a designated recipient (e.g., the caller or treasury), or
- Accumulate remainder across calls in the calling contract and periodically convert once `r` crosses `1e12`.
- Expose a dedicated administrative `sweepUSDD` function in the calling contract to transfer out stranded `USDD` dust.

If exact conservation is required for downstream accounting, document the rounding and track dust explicitly in contract state so the NAV and user accounting include or exclude it consistently.

Go back to Findings Summary

L7: Decimal truncation in swapUSDDtoUSDT leaves USDD dust and can return 0 USDT

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/libraries/SwapHelper.sol	Type:	Logic error

Description

Direct `USDD` to `USDT` conversion truncates 18 decimals to 6, leaving a remainder on the contract each call.

Function `swapUSDDtoUSDT` computes `gemAmt = usddAmount / 1e12` and calls `psm.buyGem`. Because `gemAmt` is an integer in 6 decimals, the PSM only consumes `gemAmt * 1e12` wei-USDD. Any remainder `r = usddAmount % 1e12` is stranded on the calling contract. For `usddAmount < 1e12`, `gemAmt == 0` and the function mints `0 USDT` while retaining the full `USDD` input.

Affected surface:

- Function `swapUSDDtoUSDT` in `contracts/libraries/SwapHelper.sol`.
- External call to `psm.buyGem` (confirmed by Wake `get_state_changes`).
- Token balances affected: `USDD.balanceOf(address(this))` of the calling contract accumulates dust.

Over time, repeated swaps create stranded `USDD` that must be swept or accounted for separately.

Listing 14. Code snippet from contracts/libraries/SwapHelper.sol

```
        // USDD -> USDT via PSM
        usdd.forceApprove(Constants.PSM, usddAmount);

```

```
        // When tout=0: gemAmt = usddAmount (scaled to 6 decimals)
        uint256 gemAmt = usddAmount / 1e12;

        uint256 usdtBalanceBefore = usdt.balanceOf(address(this));
        psm.buyGem(address(this), gemAmt);
        usdtAmount = usdt.balanceOf(address(this)) - usdtBalanceBefore;
```

Exploit Scenario

A user can force growth of unaccounted `USDD` dust on the contract and submit zero-output swaps.

Consider the following attack scenario:

1. Alice calls `swapUSDDtoUSDT` with amounts `usddAmount = k * 1e12 + r` for many `k`, keeping `0 < r < 1e12`.
2. The function computes `gemAmt = k` and calls `psm.buyGem` for `k` units of `USDT`.
3. Only `k * 1e12` wei-USDD are consumed; the remainder `r` remains on the contract each time.
4. Alice repeats this, inflating `USDD` dust on the contract that the library never returns.
5. Bob later finds protocol accounting skewed by contract-held `USDD` dust that requires manual sweeping or ad-hoc handling.

Small-amount griefing:

1. Alice submits `usddAmount < 1e12`.
2. The function produces `gemAmt = 0` and returns `0 USDT`.
3. The entire `usddAmount` remains on the contract as dust.

Recommendation

Add explicit dust-handling and minimum-output checks for the direct `USDD` →`USDT` path.

Fix the vulnerability by either:

- Enforcing a minimum: `require(usddAmount >= 1e12, "Insufficient amount");` or `require(gemAmt > 0, "Insufficient amount");` to reject zero-output swaps.
- Refunding or sweeping the remainder: after `psm.buyGem`, compute the contract's `USDD` balance and transfer any remainder to a designated sink (caller/treasury), or persist and batch-convert it once it exceeds `1e12`.
- Provide an administrative `sweepUSDD` function in the calling contract to recover stranded `USDD` dust on demand.

If callers expect exact conservation, augment the API to return both the `USDT` amount and any `USDD` remainder so integrators can handle it explicitly.

Go back to Findings Summary

L8: previewSUSDDNeededForUSDT ignores PSM `tout`, underestimates required sUSDD

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/libraries/SwapHelper.sol	Type:	Logic error

Description

The `previewSUSDDNeededForUSDT` path assumes a fee-less 1:1 USDD->USDT conversion. It multiplies the requested USDT by `1e12` to get `usddNeeded` and then previews how many `sUSDD` shares must be burned. When PSM `tout > 0`, `buyGem` requires extra USDD in addition to `gemAmt * 1e12`, so more `sUSDD` must be redeemed than this preview indicates.

Affected elements:

- Function `previewSUSDDNeededForUSDT` in `contracts/libraries/SwapHelper.sol`;
- External parameter `IPSM.tout` exposed by `contracts/interfaces/IPSM.sol` (not queried here).

This underestimates the sUSDD needed to satisfy a target USDT out, which can lead to shortfalls or failed swaps when used for planning.

Listing 15. Code snippet from contracts/libraries/SwapHelper.sol

```
        // USDT -> USDD needed (1:1 when tout=0)
        uint256 usddNeeded = usdtAmount * 1e12;

        // How many sUSDD shares to burn for this USDD amount
        susddAmount = susdd.previewWithdraw(usddNeeded);
```

Exploit Scenario

A realistic sequence under non-zero PSM fees:

Consider the following attack scenario:

1. Alice (governance) enables a `tout` fee on the PSM;
2. Bob uses `previewSUSDDNeededForUSDT` to determine how many `sUSDD` he must redeem to receive 10,000 USDT;
3. The preview assumes `usddNeeded = 10,000e6 * 1e12` and converts that to shares; and
4. At execution, `psm.buyGem` also charges `tout`, so additional USDD must be provided, meaning more `sUSDD` are required than previewed and the swap can under-deliver or revert.

Recommendation

Make the preview fee-aware or fail fast when the assumption is violated.

Two complementary approaches:

- Query `tout` and incorporate it into the USDD requirement before previewing shares to burn:
- `uint256 tout = IPSM(Constants.PSM).tout();`
- `uint256 usddForGem = usdtAmount * 1e12;`
- `uint256 fee = (usddForGem * tout) / 1e18;`
- `uint256 usddNeeded = usddForGem + fee;`
- `return IERC4626(Constants.SUSDD).previewWithdraw(usddNeeded);`
- Alternatively, enforce the invariant and surface the assumption explicitly:
- `require(IPSM(Constants.PSM).tout() == 0, "PSM fee enabled");`

Document rounding and consider adding a small buffer so off-chain callers do

not systematically underestimate.

Go back to Findings Summary

W1: Misleading PSM-fee assumption: deposits via sellGem still execute under tin>0 while redemptions via buyGem revert under tout>0

Impact:	Warning	Confidence:	High
Target:	./contracts/libraries/SwapHelper.sol	Type:	Code quality

Description

The library-level fee assumption claims that all swap helpers will revert if PSM fees are enabled. In reality, the USDT->sUSDD path uses `sellGem` and continues to execute under non-zero `tin` (it only receives fewer USDD), while both USDD->USDT helpers use `buyGem` and will revert under non-zero `tout` because they approve only the net USDD amount, not the required amount including the `tout` fee.

Listing 16. Code snippet from contracts/libraries/SwapHelper.sol

```
/// PSM Fee Assumption:
/// This library assumes PSM tin/tout fees are 0 (current mainnet state).
/// If PSM governance enables fees, swap functions will revert and the vault
/// will need to be upgraded. This simplifies code and saves gas.
/// Probability of fee change is low; upgrade path exists if needed.
```

Affected paths and why they behave differently:

- USDT->sUSDD path (`swapUSDTtoSUSDD`) approves USDT to the PSM gemJoin for exactly `usdtAmount` and then calls `psm.sellGem`. Under non-zero `tin`, `sellGem` transfers exactly `gemAmt` USDT and mints fewer USDD. No extra allowance is required, so the call does not revert.

Listing 17. Code snippet from contracts/libraries/SwapHelper.sol

```
// Step 1: USDT -> USDD via PSM
address gemJoin = psm.gemJoin();
```

```
usdt.forceApprove(gemJoin, usdtAmount);

// sellGem: sends USDT, receives USDD (1:1 when tin=0)
uint256 usddBalanceBefore = usdd.balanceOf(address(this));
psm.sellGem(address(this), usdtAmount);
uint256 usddReceived = usdd.balanceOf(address(this)) -
    usddBalanceBefore;
```

- USDD->USDT paths (`swapSUSDDtoUSDT` and `swapUSDDtoUSDT`) approve only the net USDD amount and then call `psm.buyGem(gemAmt)`. When `tout > 0`, `buyGem` requires `daiAmt = gemAmt * 1e12 * (WAD + tout) / WAD`, which exceeds the approved amount, causing `ERC20.transferFrom` to revert due to insufficient allowance.

Listing 18. Code snippet from contracts/libraries/SwapHelper.sol

```
usdd.forceApprove(Constants.PSM, usddReceived);

// When tout=0: gemAmt = usddReceived (scaled to 6 decimals)
uint256 gemAmt = usddReceived / 1e12;

uint256 usdtBalanceBefore = usdt.balanceOf(address(this));
psm.buyGem(address(this), gemAmt);
```

Listing 19. Code snippet from contracts/libraries/SwapHelper.sol

```
// USDD -> USDT via PSM
usdd.forceApprove(Constants.PSM, usddAmount);

// When tout=0: gemAmt = usddAmount (scaled to 6 decimals)
uint256 gemAmt = usddAmount / 1e12;

uint256 usdtBalanceBefore = usdt.balanceOf(address(this));
psm.buyGem(address(this), gemAmt);
```

The `buyGem` precondition explicitly requires approving the PSM for the amount plus the fee, which these helpers do not do when `tout > 0`.

Listing 20. Code snippet from contracts/interfaces/IPSM.sol

```
/**

 * Prerequisites:
 * - Caller must approve PSM contract for (gemAmt + fee) of DAI/USDD
 *

 * Fee calculation:
 * - fee = gemAmt * tout / WAD
 * - daiRequired = gemAmt + fee (in 18 decimals)
 */
```

Concrete fee handling in tests/mock further demonstrates the revert surface:

Listing 21. Code snippet from contracts/mocks/MockPSM.sol

```
function buyGem(address usr, uint256 gemAmt) external {
    // Calculate USDD needed: gemAmt * 1e12 * (WAD + tout) / WAD
    uint256 daiAmt = (gemAmt * 1e12 * (WAD + tout)) / WAD;

    // Transfer USDD from caller and burn
    dai.safeTransferFrom(msg.sender, address(this), daiAmt);
    MockERC20(address(dai)).burn(address(this), daiAmt);

    // Transfer USDT to user (must have been deposited via sellGem or minted)
    gem.safeTransfer(usr, gemAmt);
}
```

As a result, the header comment is misleading: enabling PSM fees does not uniformly cause all swap helpers to revert. Deposits continue to work (with reduced proceeds), while both redemption helpers will revert under non-zero `tout` because their allowance does not cover the fee. This asymmetry can mislead integrators and create one-way flows.

Exploit Scenario

A governance change enables non-zero PSM fees, and an integrator relies on the library comment to assume symmetric failure. Because the deposit path still works, assets can flow in but cannot be redeemed through the provided

helpers.

Consider the following attack scenario:

1. Alice, the integrator, keeps using `swapUSDTtoSUSDD` after PSM governance sets `tin = 1%` and `tout = 1%`;
2. Alice sends 100 USDT; `sellGem` succeeds and returns 99 USDD (due to `tin`), which are staked for sUSDD;
3. Later, Alice attempts to redeem via `swapSUSDDtoUSDT`; the helper approves only the net USDD and calls `buyGem(gemAmt)`;
4. `buyGem` requires `daiAmt = gemAmt * 1e12 * (1 + tout)`, i.e., more USDD than approved; `transferFrom` reverts;
5. A direct `swapUSDDtoUSDT` call also reverts for the same reason; and
6. Bob's protocol ends up with one-way deposits and users unable to exit via these helpers until a contract upgrade is deployed.

Recommendation

Prefer hard enforcement of the fee assumption or add explicit support for non-zero fees, to avoid asymmetric behavior.

Fix the vulnerability by either:

- Enforcing zero-fee mode: read `psm.tin()` and `psm.tout()` and revert when either is non-zero at the start of `swapUSDTtoSUSDD`, `swapSUSDDtoUSDT`, and `swapUSDDtoUSDT`. Emit a clear error (e.g., "PSM fees not supported; upgrade required"). This aligns runtime behavior with the documented assumption and prevents one-way flows; or
- Supporting fees explicitly: compute the required USDD for `buyGem` using on-chain `tout` and approve that amount instead of the net amount. For example, derive `daiRequired = gemAmt * 1e12 * (WAD + psm.tout()) / WAD` before calling `buyGem`, and add slippage bounds so callers can cap fee-

inclusive spending. Consider also handling `tin > 0` in previews and accounting paths so estimates remain accurate.

Additionally, update the library header comment to accurately describe behavior under non-zero fees (deposits continue, redemptions require fee-inclusive approvals or will revert), or add runtime guards to enforce the stated assumption.

Go back to Findings Summary

I1: Documentation mismatch: MAX_LTV is 0.915e18 (91.5%) but comment states 90%

Impact:	Info	Confidence:	High
Target:	contracts/SUSDDVault.sol	Type:	Code quality

Description

The vault caps `targetLTV` using constant `MAX_LTV`. The inline comment claims "90% max (below LLTV)", but the encoded value is `0.915e18` (91.5%). This documentation drift can mislead off-chain operators, keepers, or integrators who rely on comments and parameter runbooks rather than re-deriving the value from code. Misconfigured automation or monitoring thresholds are a common root cause of on-chain risk incidents, especially around leverage limits.

Operationally, `MAX_LTV` is enforced whenever `targetLTV` is set or changed. The discrepancy does not create a direct code execution exploit, but it reduces the perceived headroom between the configured maximum and the market liquidation threshold and can cause:

- Overly aggressive leverage targets being approved in operations based on an assumed 90% cap while the contract accepts up to 91.5%; and
- Monitoring thresholds/alerts set to 90% that fail to catch positions drifting above 90% but below 91.5%, narrowing the buffer to the market `lltv`.

Below are the exact call sites and the constant definition.

Listing 22. Code snippet from contracts/SUSDDVault.sol

```
uint256 private constant MAX_BPS = 10000;
uint256 private constant MAX_PERFORMANCE_FEE_BPS = 3000; // 30% max
uint256 private constant MAX_LTV = 0.915e18; // 90% max (below LLTV)

```

```
/// @notice Special value for targetLTV meaning "idle USDT mode" (no position)
```

The constant is consumed during initialization when validating the initial `targetLTV`.

Listing 23. Code snippet from contracts/SUSDDVault.sol

```
// Validate LTV: allow IDLE_MODE, 0 (unleveraged sUSDD), or valid leverage ratio
if (_targetLTV != IDLE_MODE && _targetLTV != 0) {
    if (_targetLTV > MAX_LTV) revert InvalidLTV();
    if (_targetLTV >= marketParams.lltv) revert LTVExceedsLLTV();
}
```

It is also used on rebalancing to validate keeper-provided `newTargetLTV`.

Listing 24. Code snippet from contracts/SUSDDVault.sol

```
// Validate LTV (allow IDLE_MODE, 0, or valid leverage ratio)
if (newTargetLTV != IDLE_MODE && newTargetLTV != 0) {
    if (newTargetLTV > MAX_LTV) revert InvalidLTV();
    if (newTargetLTV >= marketParams.lltv) revert LTVExceedsLLTV();
}
```

Exploit Scenario

Although this is not a traditional code execution bug, the mismatch can realistically lead to misoperations and loss of safety buffer.

Consider the following attack scenario:

1. Alice, operating an external keeper for the vault, assumes from the comment that the maximum allowed `targetLTV` is 90%;
2. Alice proposes or executes a rebalance to `newTargetLTV = 0.912e18` (91.2%), believing it will be rejected above 90%;
3. The contract accepts the change because `MAX_LTV` is actually `0.915e18`,

and on-chain checks pass; and

4. Bob's protocol now runs with a thinner cushion to `lltv` than intended, increasing liquidation risk under adverse price moves.

Recommendation

Fix the discrepancy so that documentation and implementation agree, and make future drift less likely:

- If 90% is the intended ceiling, change the constant to `uint256 private constant MAX_LTV = 0.90e18;`;
- If 91.5% is the intended ceiling, change the comment to "91.5% max (below LLTV)" so readers are not misled; and
- Add an invariant test that asserts the chosen ceiling is strictly below `marketParams.lltv` by the intended safety margin.

Additionally, consider deriving the limit at runtime rather than hardcoding a magic number:

- Enforce `newTargetLTV < marketParams.lltv - safetyBuffer` where `safetyBuffer` is an immutable or configurable parameter; and
- Emit an event when the effective cap is applied so off-chain systems can bind to the authoritative value.

Go back to Findings Summary

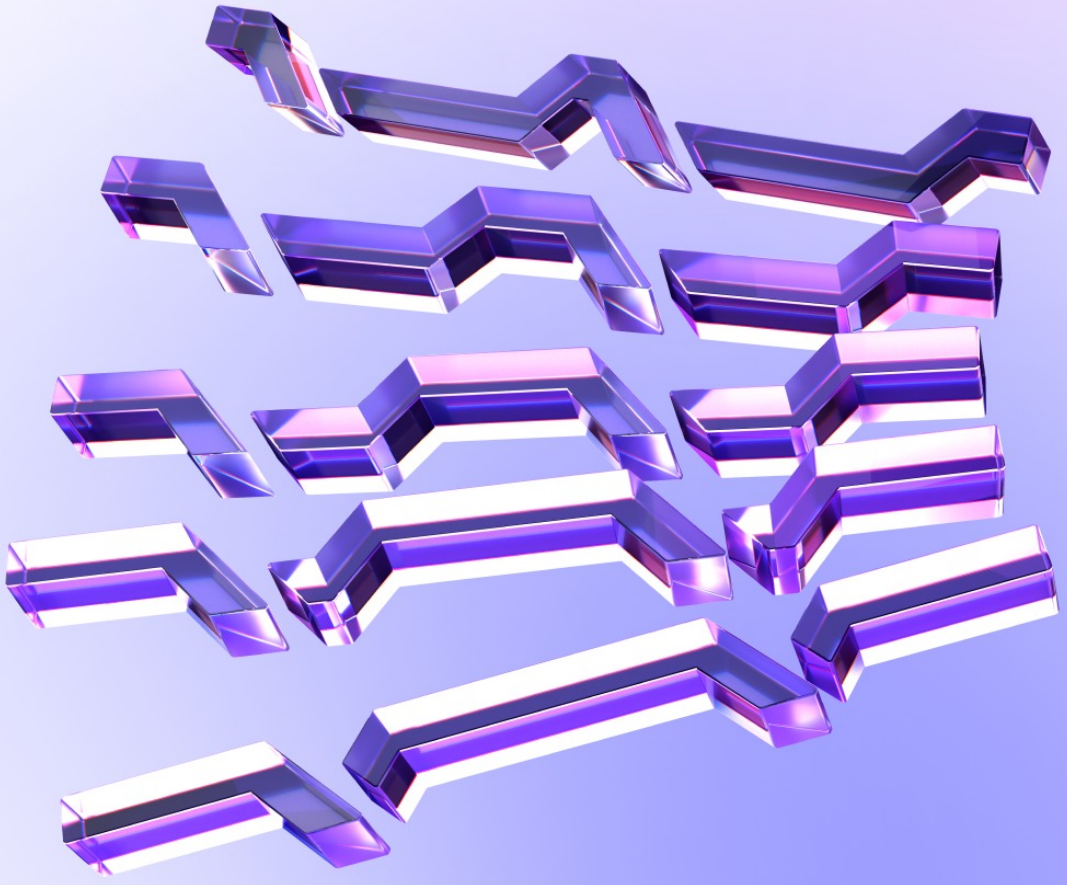
Appendix A: How to cite

Please cite this document as:

Ackee Blockchain Security, Wake Arena AI Report | evgenynacu: usdd-morpho-vault, February 25, 2026 13:22 UTC.

Thank You

Ackee Blockchain a.s.

Rohanske nabrezi 717/4
186 00 Prague
Czech Republic

hello@ackee.xyz